Chapitre 14 : Algorithmes du Plus Court Chemin

ITC - 2025/2026

Félix POTIÉ

1 - Graphe pondéré 3
2 - Algorithme de Dijkstra 7
3 - Algorithme de Bellman-Ford 27
4 - Récapitulatif 42

1 - Graphe pondéré

Dans le chapitre précédent, nous avons vu que le parcours en largeur (BFS) permettait de trouver le plus court chemin entre deux sommets en termes de **nombre d'arêtes**.

Cependant, dans la réalité, toutes les connexions n'ont pas le même "coût". Par exemple :

- Sur un réseau routier, certaines routes sont plus longues, plus embouteillées ou soumises à péage.
- Sur Internet, certains câbles ont un débit plus faible ou une latence plus élevée.

Pour modéliser ces cas, nous utilisons des **graphes pondérés**.

1 - Graphe pondéré - 2

Définition 1: Graphe pondéré.

Un graphe pondéré est un graphe où chaque arête (ou arc) possède un **poids**.
Ce poids est généralement un nombre réel ou entier.

Le **poids d'un chemin** est la somme des poids des arêtes qui le composent.

Problème du plus court chemin : Étant donné un graphe pondéré, un sommet de départ A et un sommet d'arrivée B, on cherche le chemin de A à B dont le **poids total est minimum**.

1 - Graphe pondéré - 3

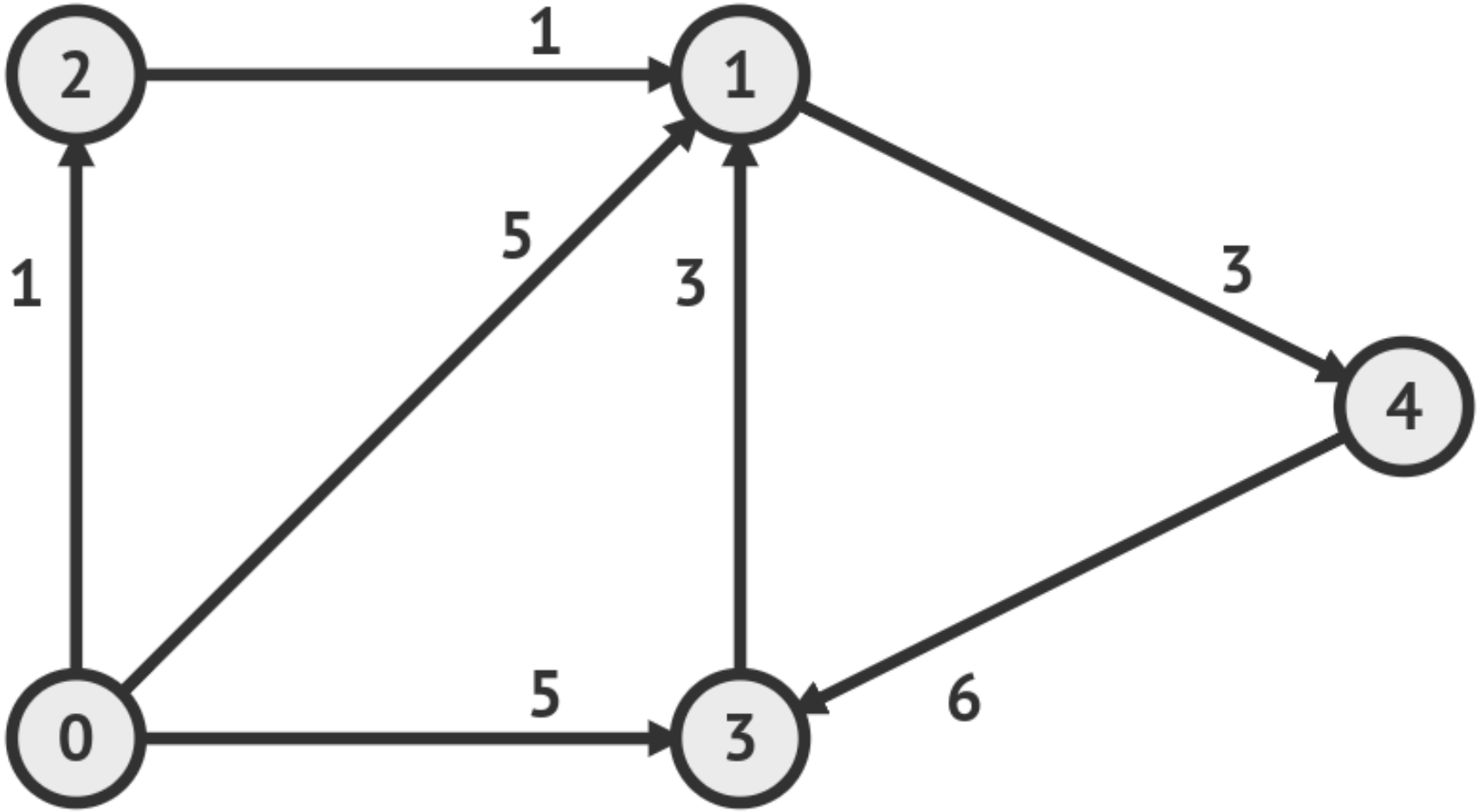


Figure 1: Exemple de graphe pondéré

1 - Graphe pondéré - 4

Comment représenter un graphe pondéré ?

- **Matrice d'adjacence :**
  On met les poids des arêtes dans la matrice (ou ∞ ou `None` si pas d'arête)

- **Liste d'adjacence :**
  On stocke des tuples (v, p) où v est un successeur de u et p le poids de l'arête $u \to v$

2 - Algorithme de Dijkstra

L'algorithme de Dijkstra (1959) est l'algorithme le plus célèbre pour résoudre ce problème, à condition que **tous les poids soient positifs ou nuls**.

Entrée :

- Graphe pondéré $G = (S, A)$
- Sommet initial s

Sortie :

- Distance la plus courte depuis s vers tous les autres sommets

2 - Algorithme de Dijkstra - 2

Définition 2: Principe de Dijkstra.

L'algorithme explore le graphe en maintenant, pour chaque sommet, la distance la plus courte connue depuis le point de départ.
À chaque étape, on sélectionne le sommet non encore visité le plus proche, et on met à jour les distances de ses voisins directs.

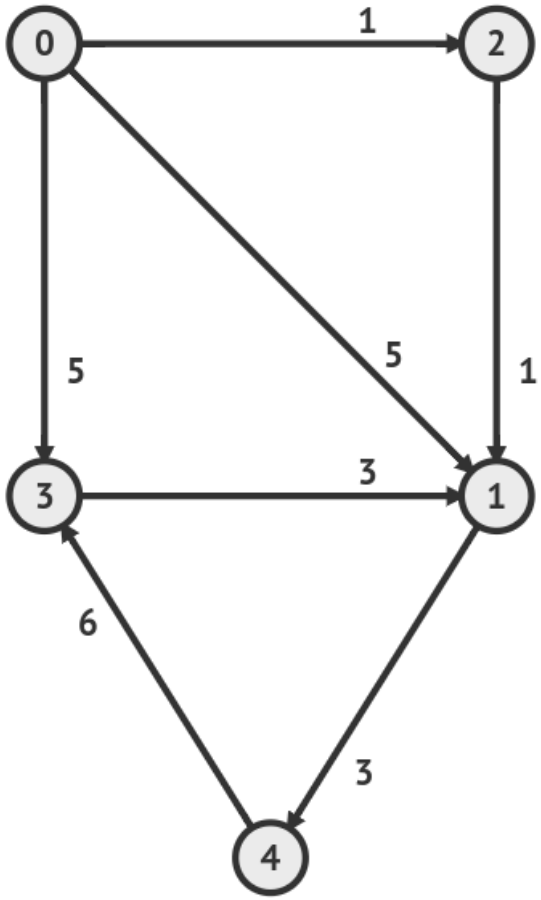
2 - Algorithme de Dijkstra - 3

Algorithme 1: Algorithme de Dijkstra.

Entrée : Graphe pondéré $G = (S, A)$, sommet initial s

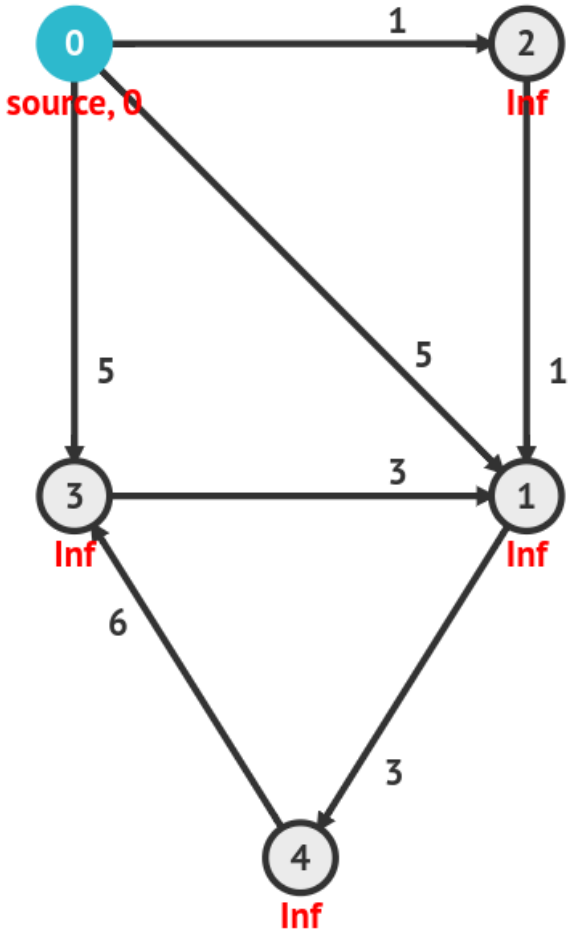
- Pour chaque sommet v : `distance`$[v]$ = $+\infty$
- `distance`$[s]$ = 0
- `a_visiter` = ensemble des sommets
- **Tant que** `a_visiter` n'est pas vide :
  - u = sommet de `a_visiter` avec `distance`$[u]$ minimale
  - Retirer u de `a_visiter`
  - **Pour chaque** voisin v de u :
    - **Si** v est dans `a_visiter` :
      - `alt` = `distance`$[u]$ + poids de l'arc $u \to v$
      - **Si** `alt` < `distance`$[v]$:
        - `distance`$[v]$ = `alt` / *On a trouvé un meilleur chemin*
- **Retourner** `distance`

2 - Algorithme de Dijkstra - 4



Dépilons l'algorithme de Dijkstra en partant du sommet 0.

2 - Algorithme de Dijkstra - 5



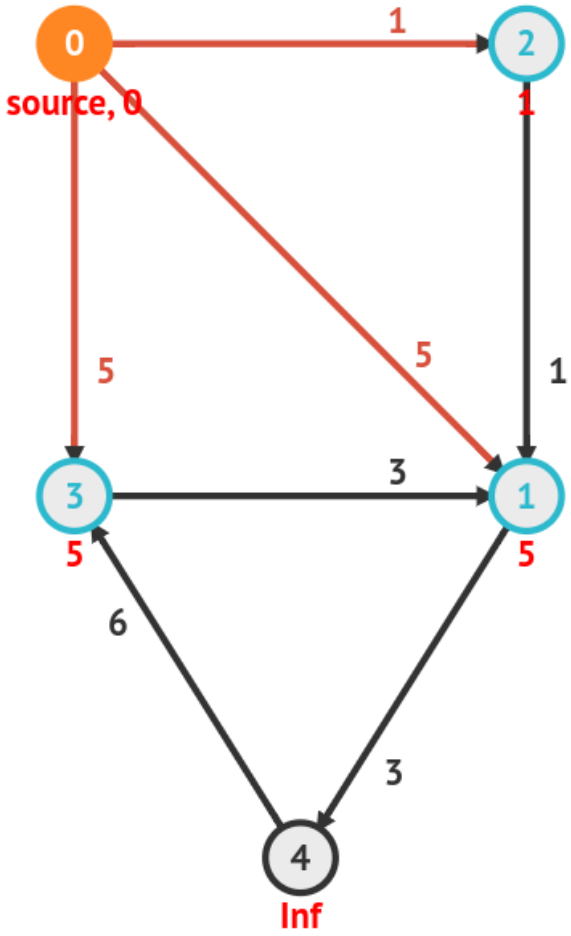
On initialise le sommet source à une distance de 0. Il est le plus petit donc on commence par lui.

`distance :`

0	1	2	3	4
0	∞	∞	∞	∞

`a_visiter:`
`{0, 1, 2, 3, 4}`

2 - Algorithme de Dijkstra - 6



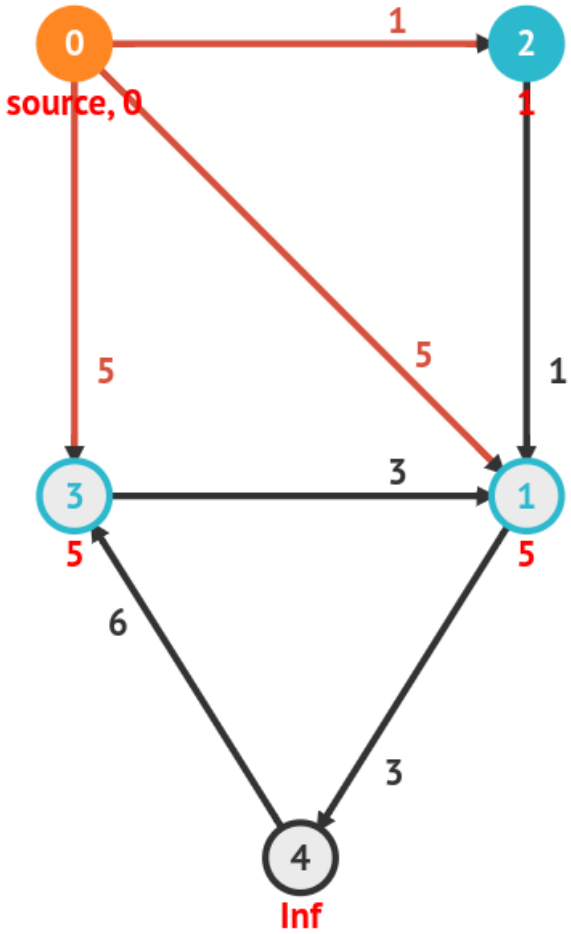
- *on enlève 0 de* `a_visiter`
- *on met à jour les distances des successeurs de 0.*

`distance :`

0	1	2	3	4
0	5	1	5	∞

`a_visiter:`
`{1, 2, 3, 4}`

2 - Algorithme de Dijkstra - 7



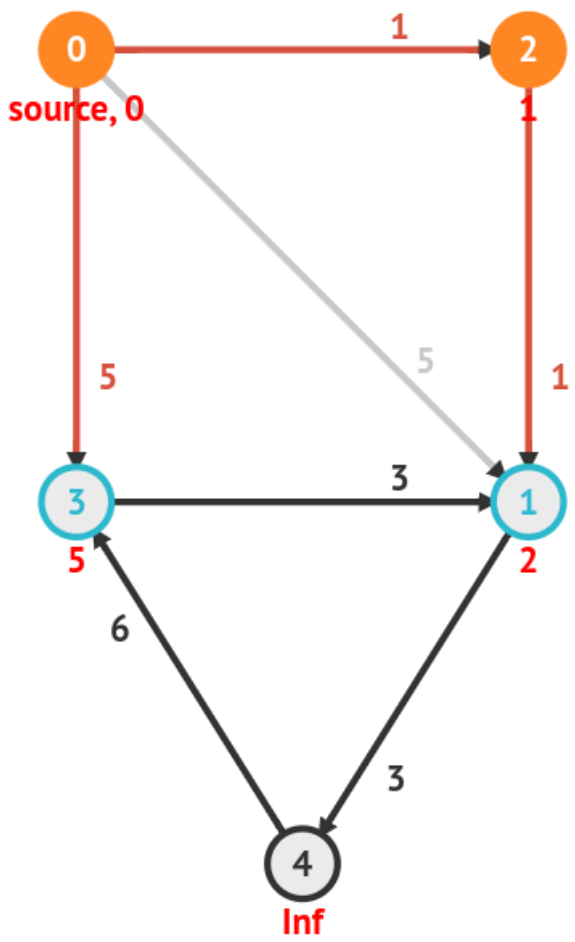
Le sommet avec une distance minimum encore dans `a_visiter` *est* 2.

`distance` :

0	**1**	**2**	**3**	**4**
0	5	1	5	∞

`a_visiter:`
`{1, 2, 3, 4}`

2 - Algorithme de Dijkstra - 8

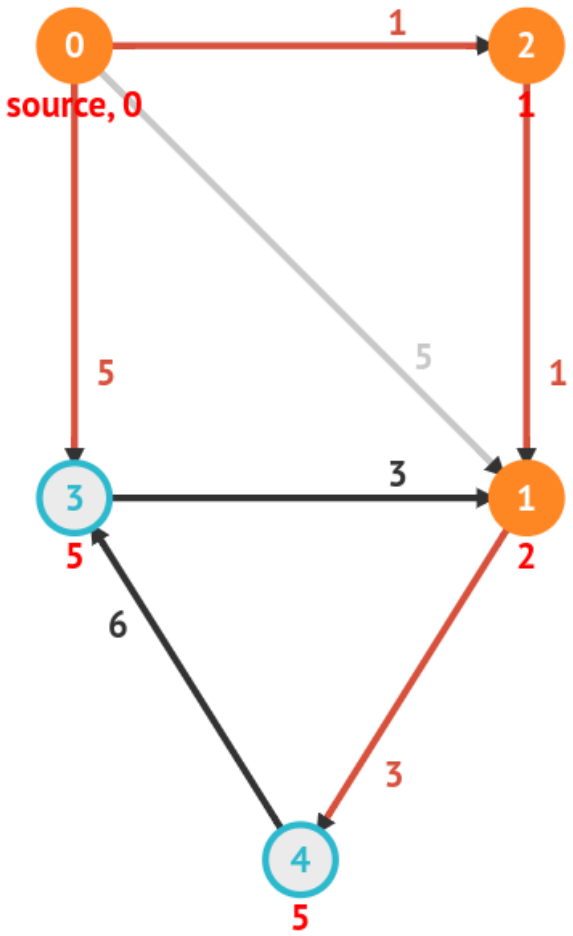


- *on enlève 2 de* `a_visiter`
- *on met à jour les distances des successeurs de 2.*
- *on peut aller de 2 à 1 (poids 1) donc de 0 à 1 en 2.*

0	**1**	**2**	**3**	**4**
0	2	1	5	∞

```
a_visiter:
{1, 3, 4}
```

2 - Algorithme de Dijkstra - 9



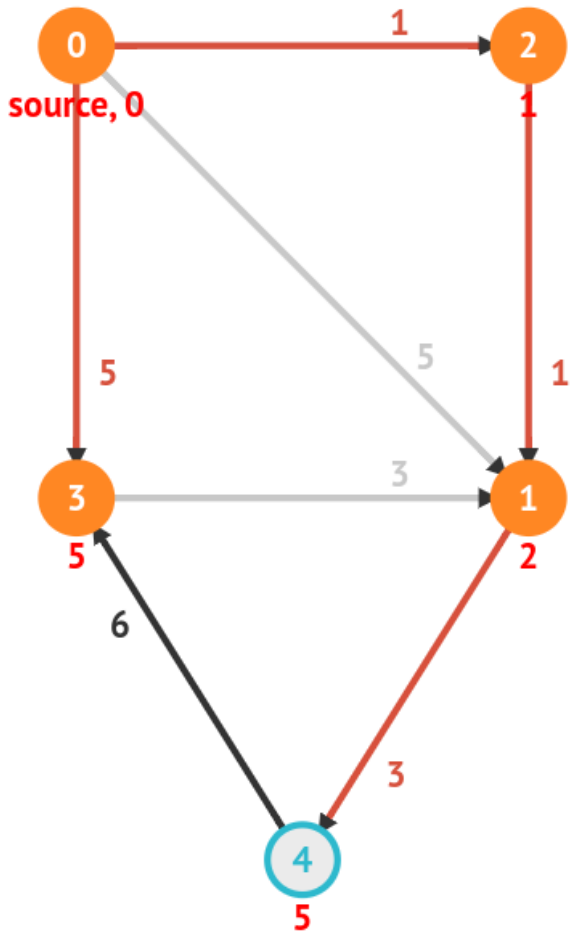
- *on enlève 1 de* `a_visiter`
- *on met à jour les distances des successeurs de 1.*

`distance :`

0	1	2	3	4
0	2	1	5	5

`a_visiter:`
`{3, 4}`

2 - Algorithme de Dijkstra - 10



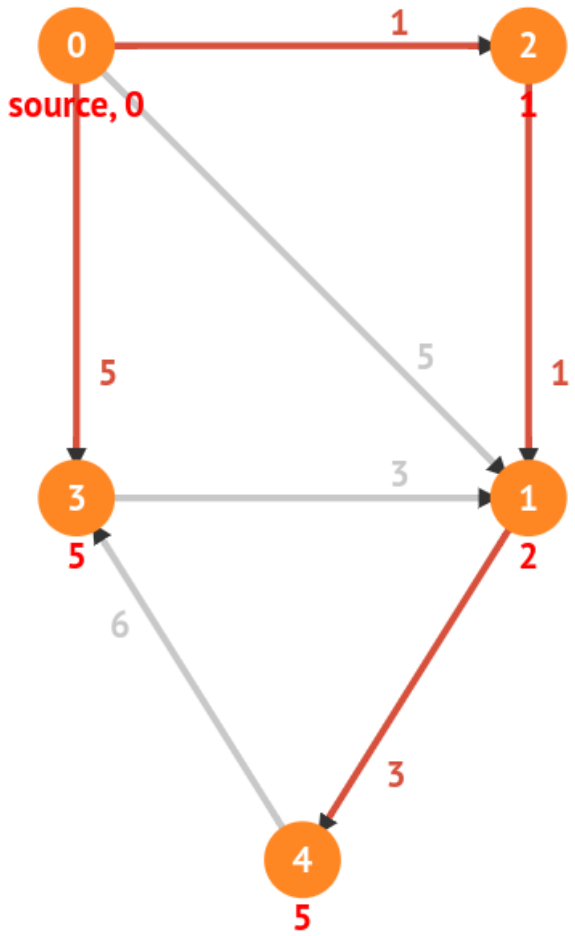
- *on enlève 3 de* `a_visiter`
- *on met à jour les distances des successeurs de 3 (aucun changement).*

`distance :`

0	1	2	3	4
0	2	1	5	5

`a_visiter:`
`{4}`

2 - Algorithme de Dijkstra - 11



- *on enlève 4 de* `a_visiter`*(seul)*
- *on met à jour les distances des successeurs de 4 (aucun changement).*

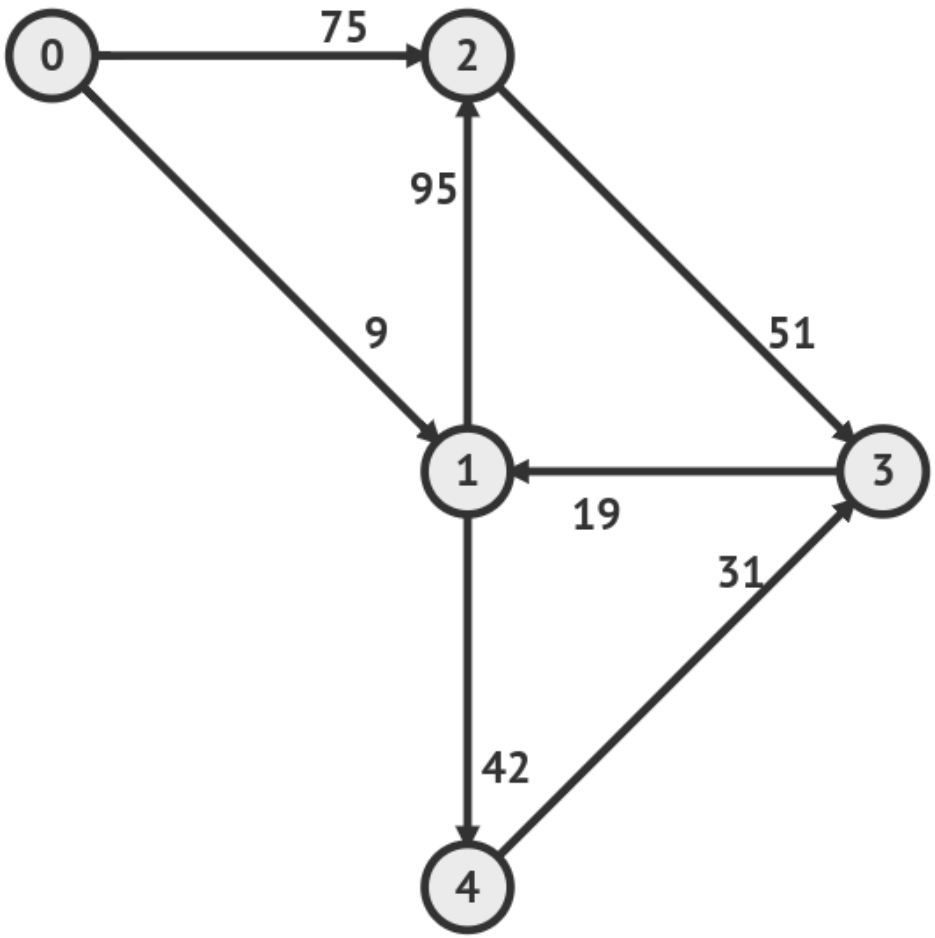
`distance` :

0	1	2	3	4
0	2	1	5	5

`a_visiter: {}`

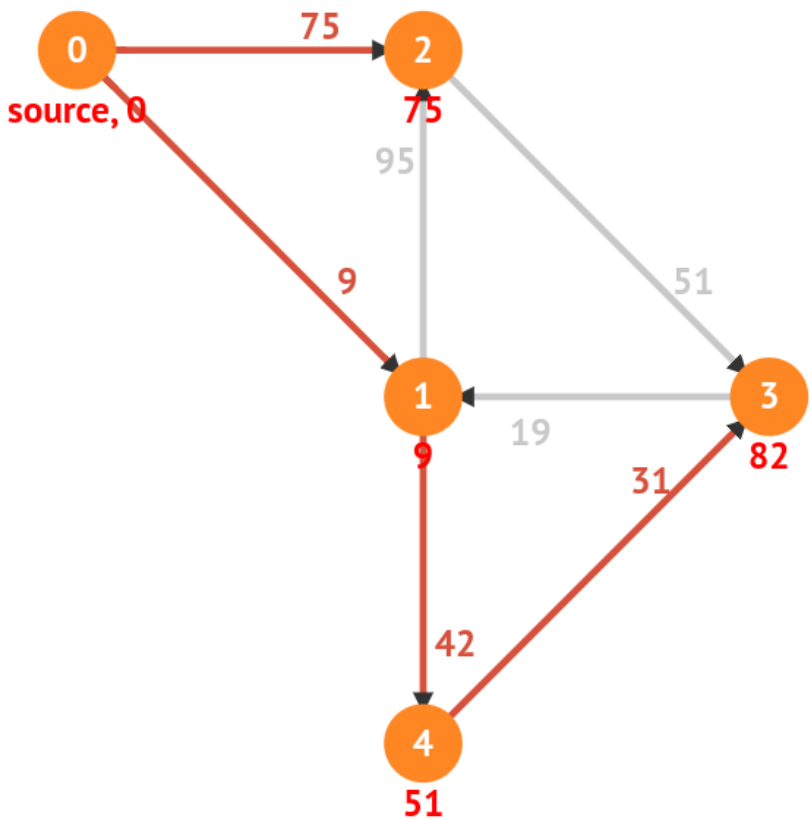
On peut donc s'arrêter.

2 - Algorithme de Dijkstra - 12



Exercice :
Dépiler l'algorithme de Dijkstra en partant du sommet 0.

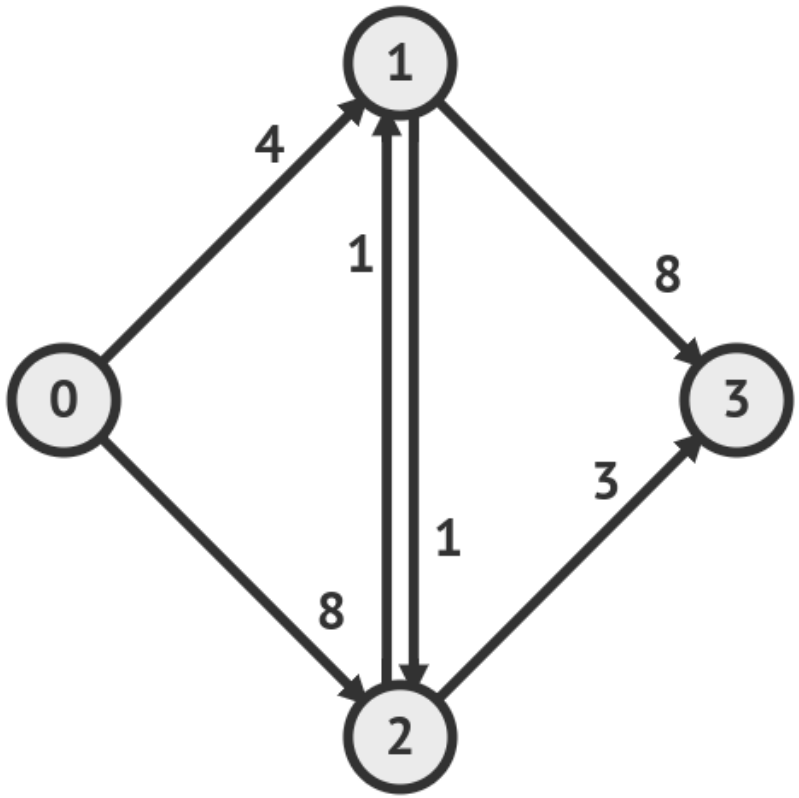
2 - Algorithme de Dijkstra - 13



distance :

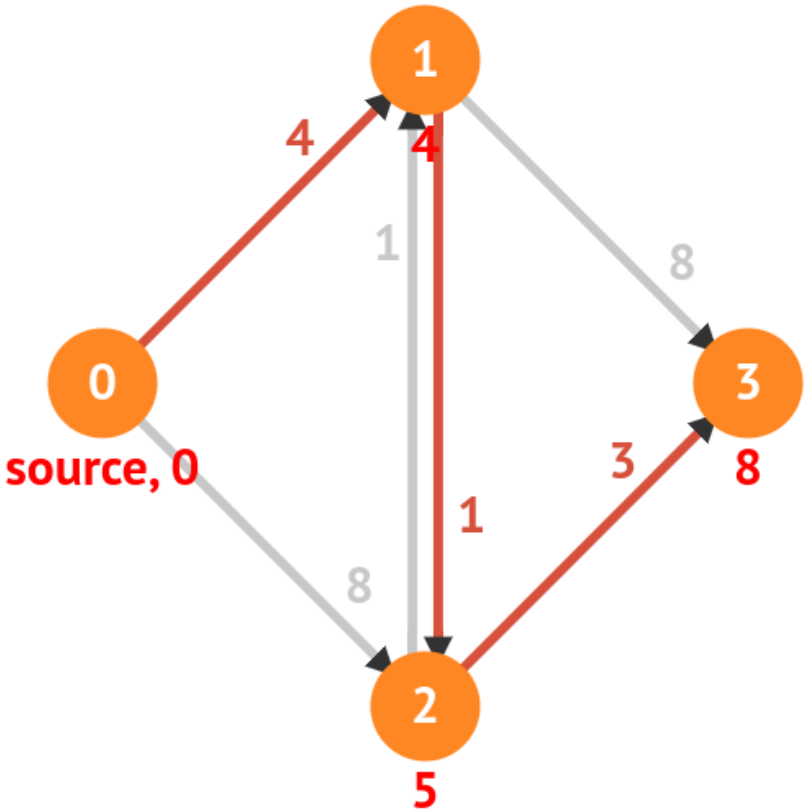
0	1	2	3	4
0	∞	∞	∞	∞
0	9	75	∞	∞
0	9	75	∞	51
0	9	75	82	51
0	9	75	82	51
0	9	75	82	51

2 - Algorithme de Dijkstra - 14



Exercice :
Dépiler l'algorithme de Dijkstra.

2 - Algorithme de Dijkstra - 15



distance :

0	1	2	3
0	∞	∞	∞
0	4	8	∞
0	4	5	12
0	4	5	8
0	4	5	8

2 - Algorithme de Dijkstra - 16

Question : De quel algorithme de parcours Dijkstra est-il une variante ?

Question : Quelle est la complexité de l'algorithme de Dijkstra ?

2 - Algorithme de Dijkstra - 17

Complexité :

- La boucle tant que tourne au plus une fois par sommet, donc on a un $O(|S|)$ multiplier par la complexité dans le boucle.
- Dans la boucle :
  - Récupérer le sommet de distance minimale avec la fonction `min()` ajoute de la complexité temporelle $O(|S|)$.
  - On parcourt les successeurs du sommet donc **au total** on aura $O(|A|)$ arcs.
    - Dans cette boucle on a que des $O(1)$

Donc la complexité de l'algorithme de Dijkstra est :
$\boldsymbol{O(|S|^2 + |A|)}$.

2 - Algorithme de Dijkstra - 18

Problème de performance : on se rend compte que de retrouver le sommet de distance minimale avec la fonction `min()` ajoute de la complexité temporelle.

Définition 3: Amélioration avec file de priorité.

Pour identifier rapidement le sommet le plus proche lors de l'exploration, l'algorithme utilise généralement une structure de données optimisée appelée **file de priorité**.

Nous utiliserons le module Python `heapq`.

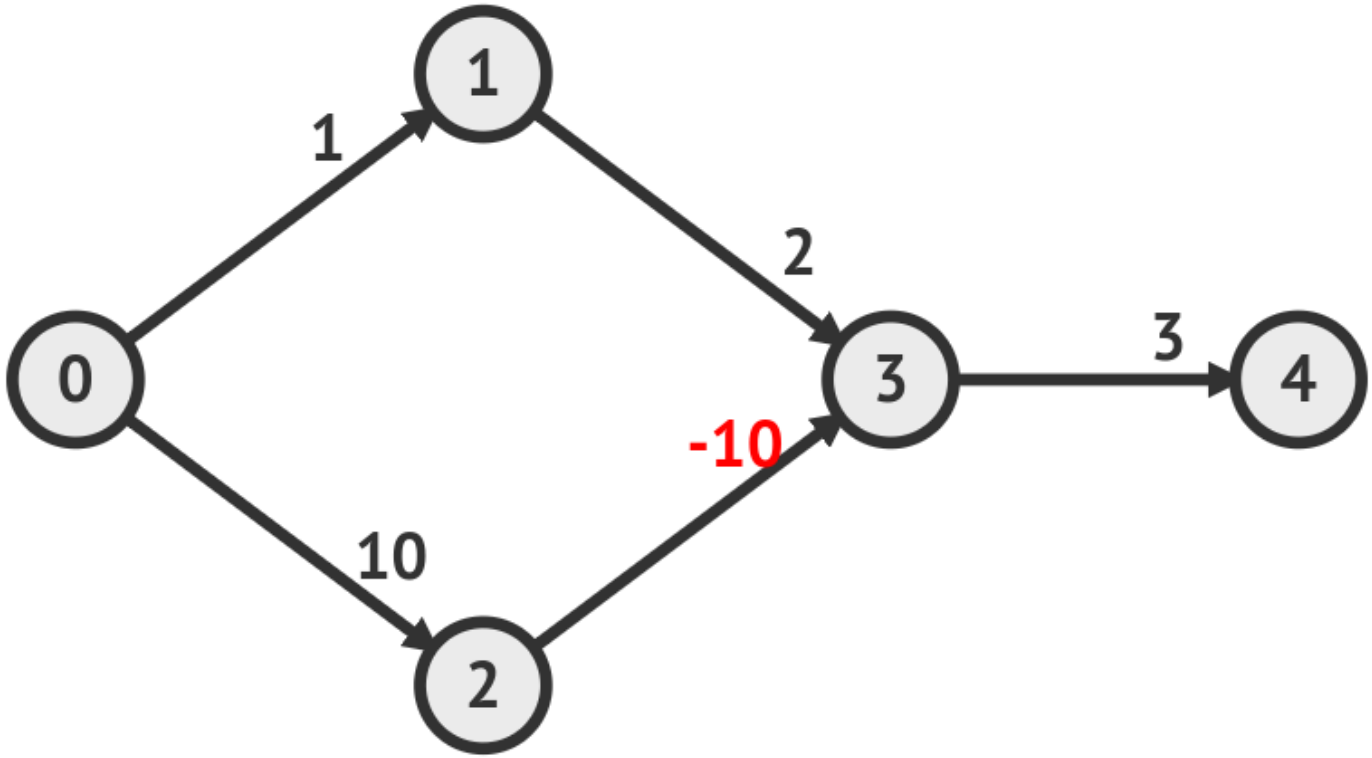
- insertion dans file de priorité : $O(\log|S|)$
- extraction du min de la file de priorité : $O(\log|S|)$

On a maintenant une complexité de $\boldsymbol{O((|S| + |A|)\log|S|)}$.

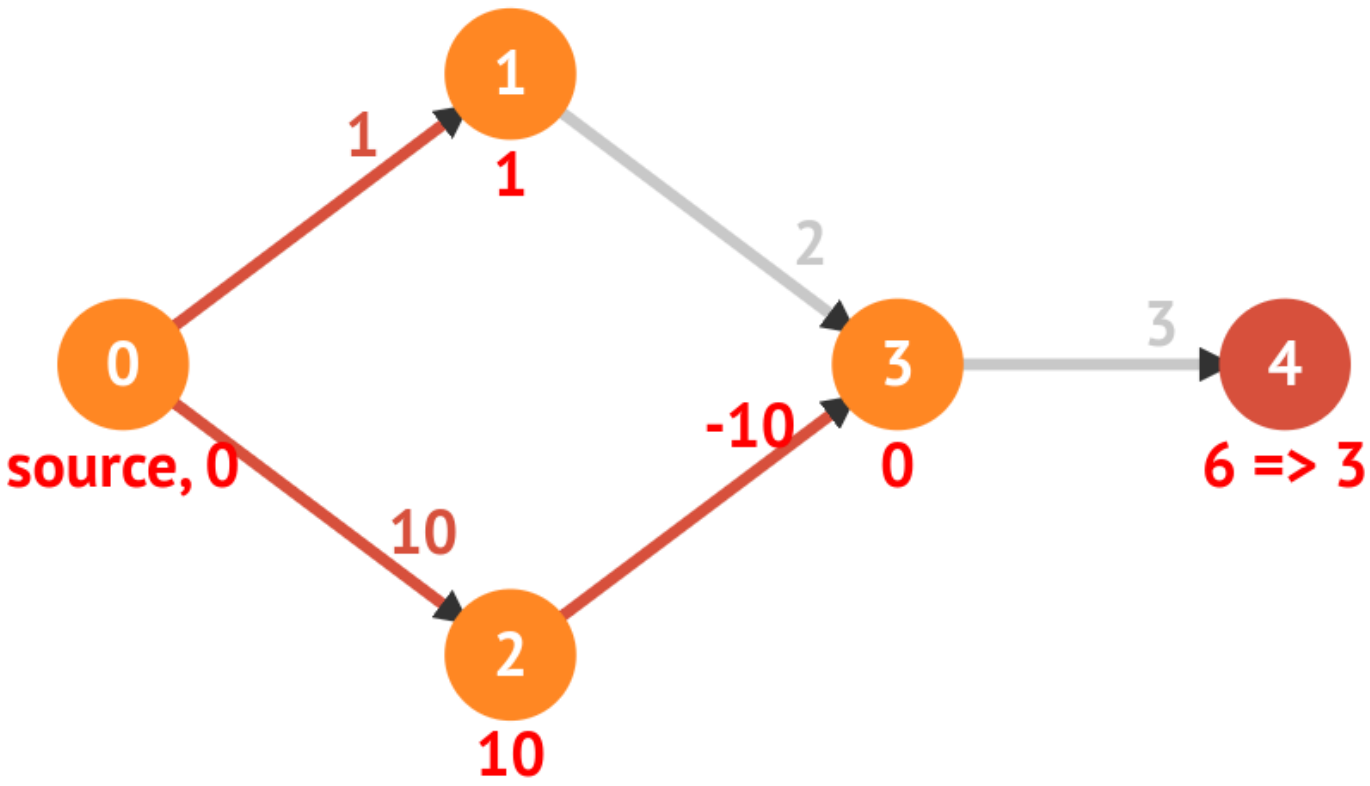
2 - Algorithme de Dijkstra - 19

Que se passe-t-il si notre graphe possède des poids négatifs ?

Regardons sur cet exemple :



2 - Algorithme de Dijkstra - 20



Ici l'algorithme de Dijkstra nous dit que le plus court chemin de 0 vers 4 est de longueur 6. Alors qu'on peut trouver un chemin de longueur 3 en passant par 2.

3 - Algorithme de Bellman-Ford

L'algorithme de **Bellman-Ford** (1958) permet de trouver le plus court chemin dans un graphe avec des poids négatifs. Il y a tout de même une contrainte : il ne peut pas gérer les cycles de poids négatif.

Définition 4: Principe de Bellman-Ford.

Au lieu de choisir intelligemment le prochain sommet à visiter, on met à jour **toutes les arêtes** du graphe, et on répète cette opération tant qu'on a fait au moins une mise à jour.

3 - Algorithme de Bellman-Ford - 2

L'algorithme de Bellman-Ford se base sur le principe de Programmation Dynamique.

Soit $G = (S, A)$. S sommets, A arcs et w(u,v) le poids de l'arc de $u \to v$ (∞ si aucun arc).

1. On décompose le problème en sous-problème $D^k(t)$ qui est la plus courte distance du sommet s à t avec un chemin qui contient au plus k arcs.
2. On a la relation de récurrence :
   - $D^k(s) = 0$
   - $D^0(t) = +\infty$ pour $t \neq s$
   - $D^k(t) =$min $\left\{D^{k-1}(t), \min_{u-t}\left(D^{k-1}(u) + w(u,t)\right)\right\}$

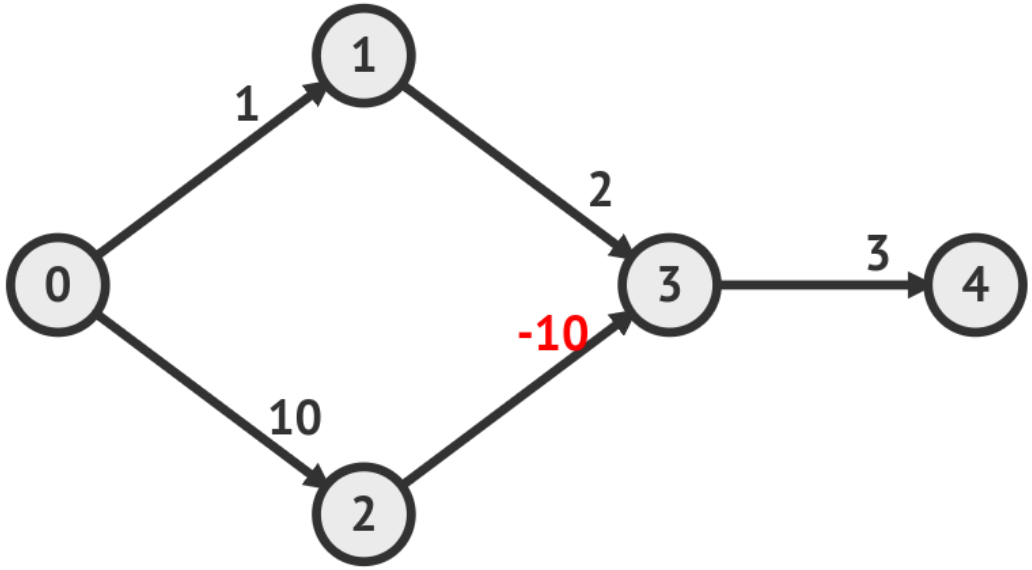
3 - Algorithme de Bellman-Ford - 3

Algorithme 2: Algorithme de Bellman-Ford.

Entrée : Graphe $G = (S, A)$, sommet initial `s`

- Pour chaque `v` dans S : `distance[v]` = $+\infty$
- `distance[s]` = 0
- `changement` = Vrai
- **Tant que** `changement` est vrai :
  - `changement` = Faux
  - `nouvelle_distance` = copie de `distance`
  - **Pour chaque** arête (u, v) de poids p :
    - **Si** `distance[u] + p` < `distance[v]` :
      - `nouvelle_distance[v]` = `distance[u] + p`
      - `changement` = Vrai
  - `distance` = `nouvelle_distance`
- Retourner `distance`

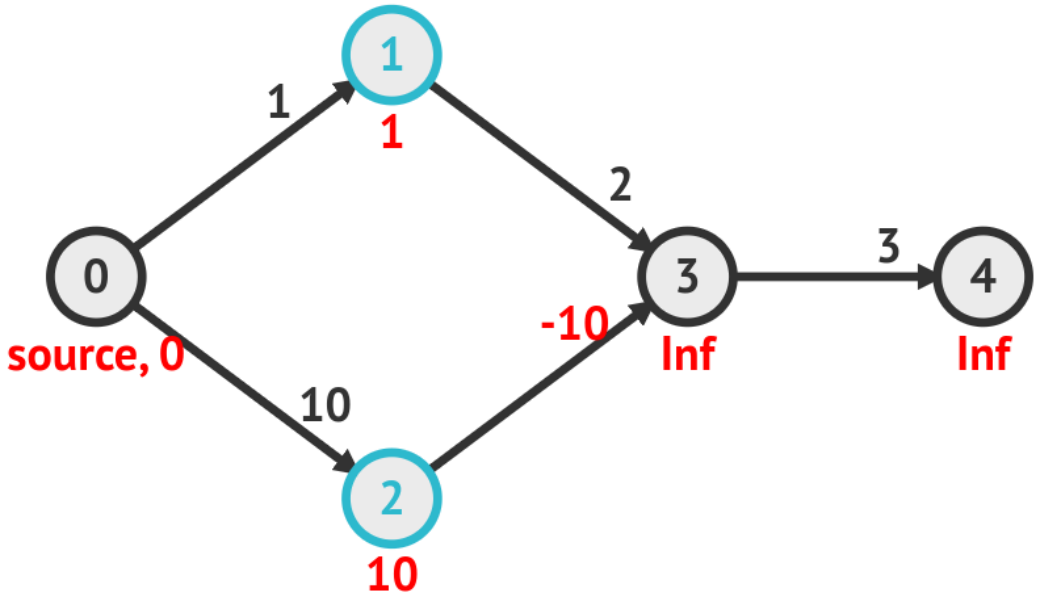
3 - Algorithme de Bellman-Ford - 4



distance :

0	1	2	3	4
0	∞	∞	∞	∞

3 - Algorithme de Bellman-Ford - 5

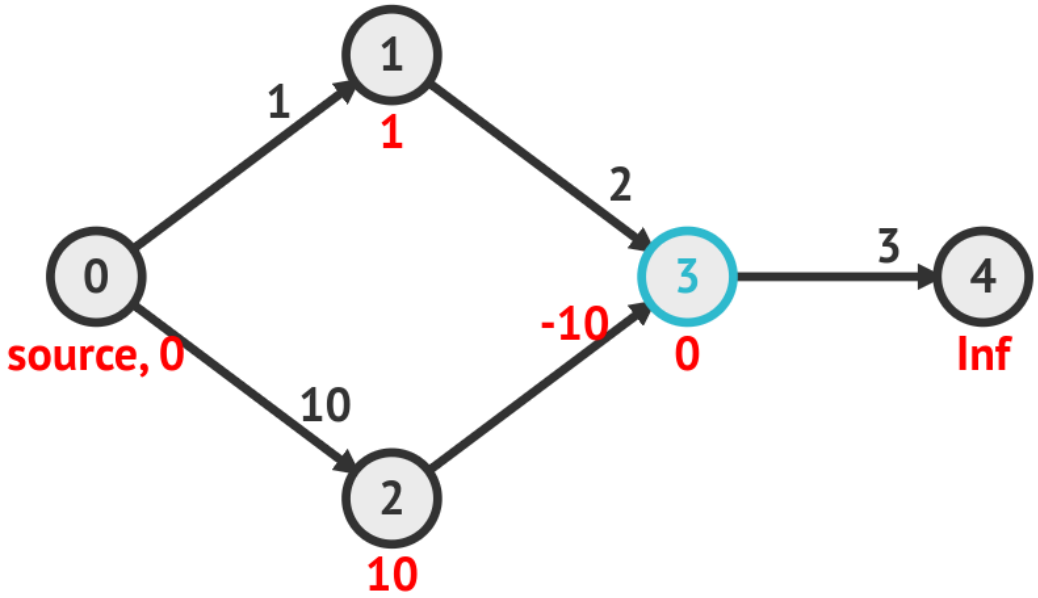


`distance` :

0	1	2	3	4
0	1	10	∞	∞

On a changé la valeur de 1 et 2, donc on continue.

3 - Algorithme de Bellman-Ford - 6

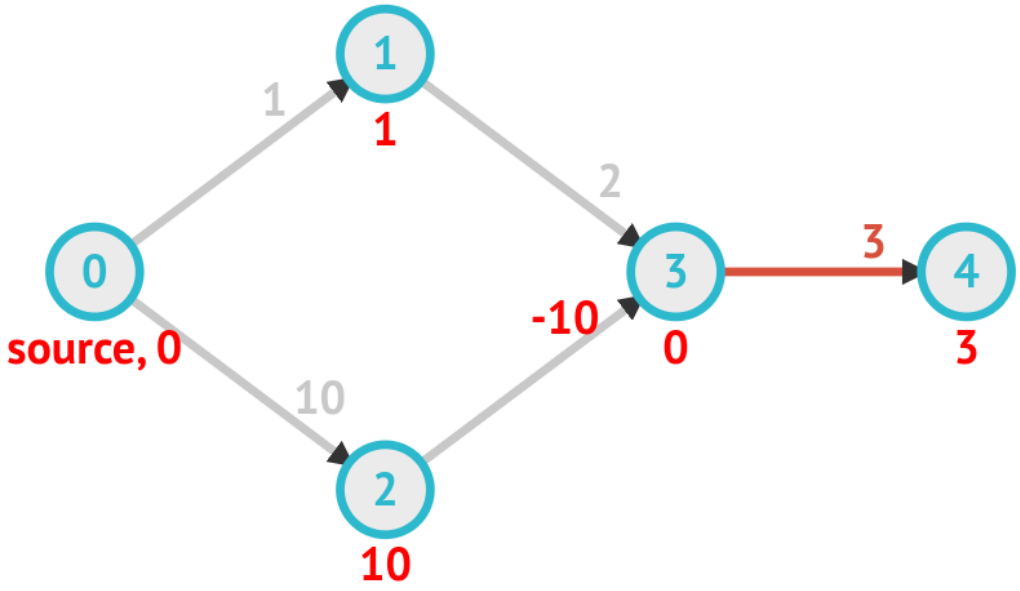


`distance` :

0	1	2	3	4
0	1	10	0	∞

On a changé la valeur de 3, donc on continue.

3 - Algorithme de Bellman-Ford - 7

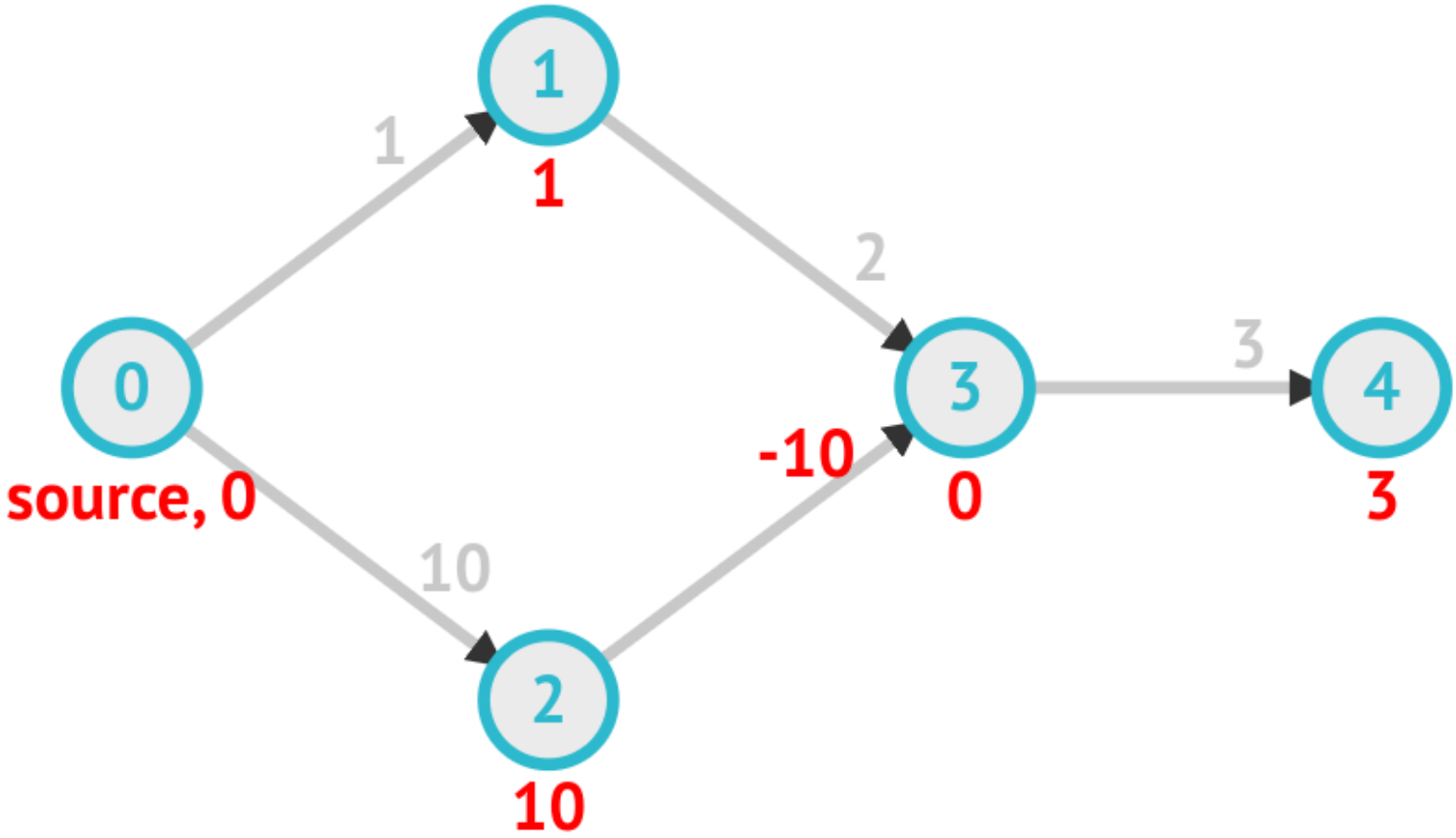


`distance` :

0	1	2	3	4
0	1	10	0	3

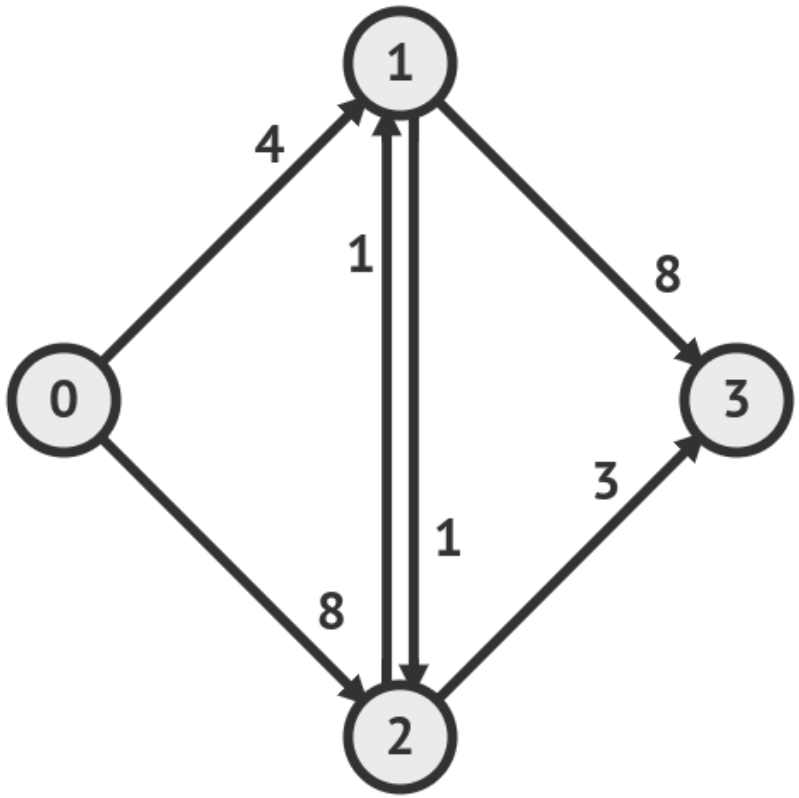
On a changé la valeur de 4, donc on continue.

3 - Algorithme de Bellman-Ford - 8



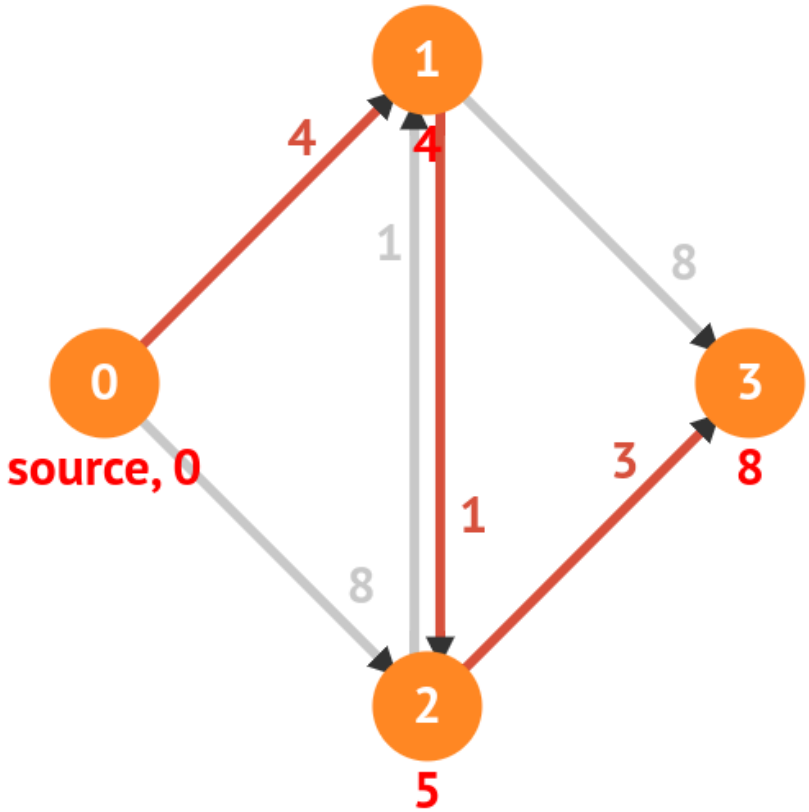
Plus aucun changement, on peut donc s'arrêter.

3 - Algorithme de Bellman-Ford - 9



Exercice :
Dépiler l'algorithme de Bellman-Ford.

3 - Algorithme de Bellman-Ford - 10



0	1	2	3
0	∞	∞	∞
0	4	8	∞
0	4	5	12
0	4	5	8
0	4	5	8

On trouve bien le même résultat qu'avec Dijkstra

3 - Algorithme de Bellman-Ford - 11

Remarque : Au maximum, l'algorithme de Bellman-Ford tel qu'il est présenté ci-dessus effectue $|S| + 1$ tours.
On pourrait donc s'arrêter dès qu'on a fait $|S|$ tours.

Question : Quelle est la complexité de l'algorithme de Bellman-Ford ?

3 - Algorithme de Bellman-Ford - 12

- Boucle while : $|S|$ tours:
  - copie de distance : $O(|S|)$
  - On parcourt toutes les arêtes avec la boucle for : $O(|A|)$

Complexité : $\boldsymbol{O(|S| * (|S| + |A|)) = O(|S|^2 + |S| * |A|)}$

On se rend compte que la copie de distance augmente la complexité et n'est pas nécessaire, on peut directement modifier la distance, notre algorithme ira plus vite.

La complexité de cette version optimisé sera donc $\boldsymbol{O(|S| * |A|)}$.

3 - Algorithme de Bellman-Ford - 13

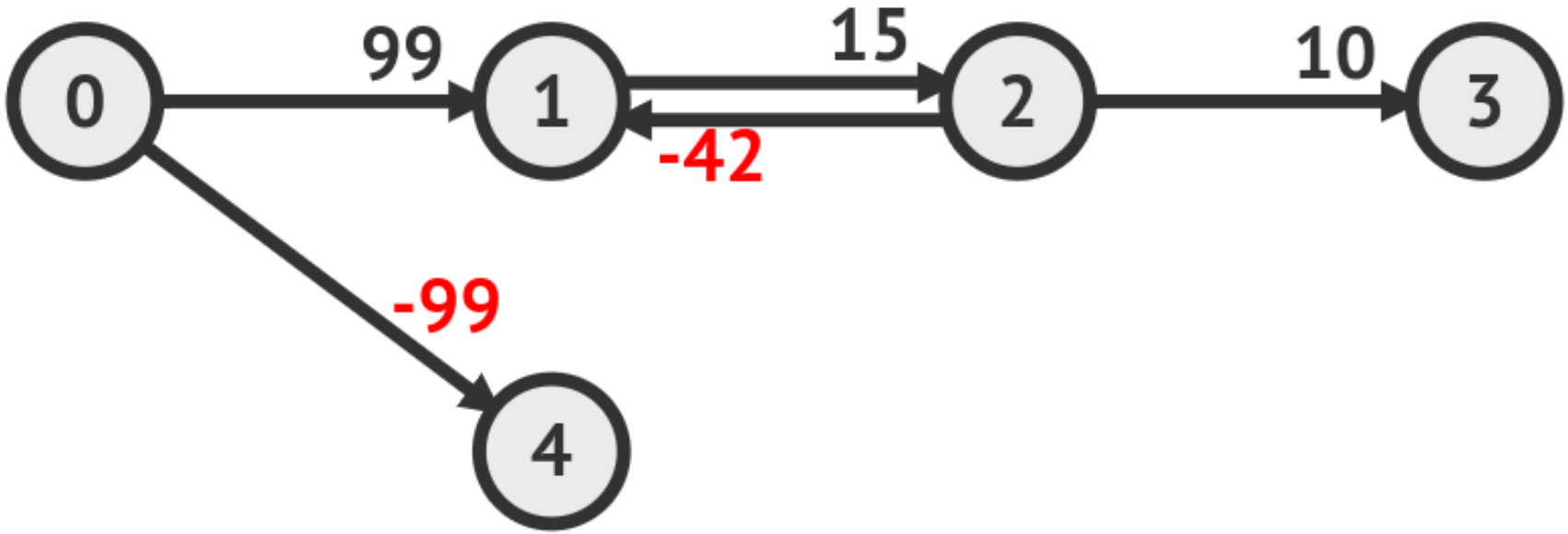
Algorithme 3: Algorithme de Bellman-Ford (optimisé).

Entrée : Graphe $G = (S, A)$, sommet initial `s`

- Pour chaque `v` dans S : `distance[v]` = $+\infty$
- `distance[s]` = 0
- `changement` = Vrai
- **Tant que** `changement` est vrai :
  - `changement` = Faux
  - **Pour chaque** arête (u, v) de poids p :
    - **Si** `distance[u] + p` < `distance[v]` :
      - `distance[v]` = `distance[u] + p`
      - `changement` = Vrai
- Retourner `distance`

3 - Algorithme de Bellman-Ford - 14

Question : Pourquoi on ne peut pas utiliser Bellman-Ford sur un graphe qui contient des cycles de poids négatif ?



0	1	2	3	4

3 - Algorithme de Bellman-Ford - 15

0	∞	∞	∞	∞
0	99	∞	∞	-99
0	99	114	∞	-99
0	57	114	124	-99
0	57	72	124	-99
0	15	72	82	-99

On se rend compte que la distance des sommet 1,2 et 3 continue de diminuer, c'est à cause du cycle de poids négatif. L'algorithme ne termine jamais.

4 - Récapitulatif

Visualisation des algorithmes : https://visualgo.net/en/sssp

Dijkstra	Bellman-Ford
• **Contraintes :** Arêtes de poids positifs seulement • **Algorithme :** Variante du parcours en largeur • **Complexité :** $O((\|S\| + \|A\|) \log\|S\|)$ • **Utilisation :** GPS (amélioration A*), Routage réseau OSPF	• **Contraintes :** Pas de cycle de poids négatif • **Algorithme :** Programmation dynamique • **Complexité :** $O(\|S\| \cdot \|A\|)$ • **Utilisation :** Détection de cycle de poids négatif, Routage réseau RIP